

Lab 2 Report

In this lab, two versions of assembly programs capable of calculating the first $N+1$ Recaman numbers given N are implemented: recursive and iterative. The differences in their performance are discussed in this report.

The recursive algorithm contains a function that calls itself until it reaches a base case and thus calculates the numbers in the sequence in a top-down approach. The assembled program for the recursive implementation uses 224 bytes. The total number of executed instructions is 1197, and the total number of memory accesses is $1197 + 296 + 207 = 1700$, as seen in Figure 1a). The iterative algorithm does not have a function that calls itself. Instead, it is made up of a loop, which will calculate the numbers in the sequence in a bottom-up approach. The assembled program for the iterative implementation uses 208 bytes. The total number of executed instructions is 1002, and the total number of memory accesses is $1002 + 171 + 82 = 1225$, as seen in Figure 1b).

Counters		Counters	
Counter	Value	Counter	Value
Exec. instructions	1197	Exec. instructions	1002
Data loads	296	Data loads	171
Data stores	207	Data stores	82
Simulator run time (ms)	2	Simulator run time (ms)	2
Simulator core run time (ms)	1	Simulator core run time (ms)	1
Simulated MIPS	1.197	Simulated MIPS	1.002

Figure 1. Performance data from Counters pane for Recaman assembly programs with $N=20$.
1a) Recursive version (left). 1b) Iterative version (right).

We notice that the iterative version performs significantly better than the recursive one, in all three benchmarks. With the $N=20$ example, we observe a 39% increase in the number of memory accesses in the recursive program compared to the iterative program, which is quite significant. This could be because for each recursive call, there is more setup that needs to be repeated, like saving the function arguments into variable registers. Notably, the number of data stores in the recursive program is more than 2.5 times higher than that of the iterative program. This makes sense, because since the recursive program must call itself top-down until it reaches a base case, it must store the pending values in the stack while waiting for the recursive calls to return, which will result in an increase in data stores and loads. On the other hand, the iterative program does not need to push pending operations to the stack (other than the search calls, like the recursive one) and directly takes the result from the previous iteration to build off of, making it require less memory accesses. Finally, the file size is comparable: the iterative version takes only 16 bytes less. This means the recursive program required a few more instructions, but this could also be due to inefficient implementation from the programmer.

However, we can repeat this experiment with a higher number N as input. With $N=30$, the assembled program for the recursive implementation uses 232 bytes. The total number of executed instructions is 2255, and the total number of memory accesses is $2255 + 3079 + 307 = 3079$. The assembled program for the iterative implementation uses 216 bytes. The total number of executed instructions is 1960, and the total number of memory accesses is $1960 + 332 + 122 = 2414$. This represents a 27.5% increase in memory accesses for the recursive program compared to the iterative one, which is a smaller increase than $N=20$. This could be because the more operations need to be performed for both, the less the difference grows. With $N=40$, the recursive program fails to execute, due to function calls nested too many levels. The iterative program succeeds, which shows that recursion is limited to the stack size.

In general, this implies that although recursive implementations may sometimes be more elegant in terms of the written solution, they are more resource-intensive and less efficient. Moreover, the performance differences between the two implementations becomes even more apparent as the input argument gets bigger, where recursion may even fail.